Input Pipeline: Technical Documentation

1. Overview

The Input Pipeline is the first stage of the Export Intelligence platform. Its job is to take a company's website URL, crawl it intelligently, extract structured product and company data using AI, and score each product for data quality.

This pipeline runs before any intelligence modules. The output feeds directly into the intelligence engine (Modules 1–6) which generates market insights, buyer lists, trade data, and marketing content.

What It Does

Step	Stage	What Happens
1	Crawl	Playwright browser visits the website, discovers and fetches all product, about, and contact pages
2	Clean	HTML noise (nav, footer, ads) is stripped. Clean text is extracted using trafilatura + BeautifulSoup fallback + live browser text for React/SPA sites
3	Classify	Each page is classified as: PRODUCT, CATEGORY, ABOUT, CONTACT, HOME, BLOG, or IGNORE based on URL patterns

		and page content signals
4	Extract	GPT-4o-mini reads the clean text and extracts structured JSON: product name, category, price, HS code, certifications, MOQ, packaging, ingredients, images
5	Score	Each product gets a confidence score 0–100 based on which fields were found. Semantic scoring (LLM) then adjusts scores in the 40–85 range
6	Return	Results returned via FastAPI endpoint as structured JSON ready for DB insertion

2. Architecture

File Structure

All pipeline code lives at:

`~/Desktop/AI/website_dna/input_pipeline/`

File / Folder	Responsibility

models/page.py	Data containers: RawPage → CrawledPage → ClassifiedPage
models/product.py	ProductData model — maps to product_master DB table
models/company.py	CompanyData model — maps to company_preference DB table
config.py	All settings in one place: crawler limits, LLM models, scoring weights, cache TTLs
crawler/browser.py	Playwright browser session — fetches pages, handles React/SPA sites, clicks accordions
crawler/link_extractor.py	Finds and scores links on each page using URL patterns
crawler/orchestrator.py	BFS crawl loop — ties all crawler components together
classifier/url_classifier.py	Classifies URLs by pattern BEFORE fetching (no network cost)
classifier/content_classifier.py	Confirms page type AFTER fetching using content signals
cleaner/content_cleaner.py	Strips HTML noise, extracts clean text for LLM
extractor/prompts.py	All LLM prompts as versioned constants
extractor/product_extractor.py	Calls GPT-4o-mini to extract structured product JSON
extractor/company_extractor.py	Merges about/contact pages, extracts company JSON
scorer/confidence_scorer.py	Rule-based field presence scoring (fast, no LLM)

scorer/semantic_scorer.py	LLM quality scoring for mid-confidence products (40–85 range only)
pipeline.py	End-to-end orchestration of all 5 stages
run.py	CLI entry point — run from terminal
api.py	FastAPI endpoint — POST /pipeline/run for backend integration

Data Flow

Data moves through 3 containers: RawPage (browser output) → CrawledPage (after cleaning) → ClassifiedPage (after classification). These are Pydantic models defined in models/page.py.

RawPage url, path, title, html, inner_text, depth, status, error	→	**CrawledPage** url, path, title, depth, clean_text, snippet, word_count, error	→	**ClassifiedPage** + page_type, type_confidence, classified_by	→	**Product Data** Full extracted + scored product

3. The Crawler

How It Works

The crawler uses BFS (Breadth-First Search) — it starts at the homepage and fans outward level by level. This ensures important pages close to the homepage (depth 1–2) are fetched before deep nested pages.

Setting	Default	What It Controls
max_depth	3	Maximum levels deep from homepage
max_pages	500	Hard cap — stops crawling after this many pages total
page_timeout_ms	20,000	Give up on a page after 20 seconds
wait_after_load	5,000 ms	Extra wait after page load — gives JavaScript time to render content
concurrency	3	Max simultaneous page fetches — keeps us under bot detection thresholds

Depth Budgets

Different page types get different crawl depth allowances. This prevents wasting crawl budget on blogs or login pages:

Page Type	Max Depth	Reason

PRODUCT	3	Product pages can be nested deep: Home → Shop → Category → Product
CATEGORY	2	Crawled to find product links inside, not for extraction
ABOUT	1	Always 1 click from homepage
CONTACT	1	Always 1 click from homepage
HOME	1	Just extract links from it, don't go deeper
BLOG / IGNORE	0	Skip entirely — not relevant for product intelligence

React / SPA Site Handling

Many modern B2B websites are built with React, Vue, or Angular. These sites send an empty HTML shell initially — the actual content is built by JavaScript in the browser. Our crawler handles these specially.

The crawler uses three techniques for SPA sites:

document.documentElement.outerHTML instead of page.content() — reads the fully JS-rendered DOM, not the server-sent shell

Wait for body text length > 500 chars AND no redirect text — ensures React has fully mounted before grabbing HTML

inner_text() passed directly to the cleaner — bypasses trafilatura which struggles with React's deeply nested div structure

Resource Blocking

To speed up crawling, the browser blocks some resource types:

BLOCKED: images, media files, fonts — these are heavy and not needed for text extraction

ALLOWED: stylesheets — required for React/Vite sites where CSS imports trigger JS module loading

Blocking stylesheets was initially added for speed but caused React sites to stop rendering. Stylesheets are now always allowed.

Skip Patterns

URLs containing any of these strings are skipped entirely without fetching:

Auth: login, signin, signup, register, logout, account, password

Commerce noise: cart, checkout, wishlist, order, invoice

Legal: terms, privacy, cookie, sitemap, disclaimer, refund

Static assets: .pdf, .jpg, .png, .gif, .css, .js, cdn, wp-content

Support: faq, support, help, 404, 403

Protocols: #, mailto:, tel:, javascript:, whatsapp:

Irrelevant: blog, news, gallery, testimonial, career, jobs, search

4. Page Classification

Two-Stage Classification

Classification runs in two stages to balance speed and accuracy:

Stage	When	How

URL Classification	Before fetching	Checks URL path against signal lists. No network cost. Fast. Can skip pages before fetching them.
Content Classification	After fetching	Checks actual page text against content signals. Confirms or overrides URL classification. Catches edge cases.

Classification Priority Order

CATEGORY is checked BEFORE PRODUCT in URL classification. This is critical — /product-category/energy contains the word 'product' as a substring. Without this ordering, category pages get misclassified as products.

URL classifier checks in this exact order (first match wins):

1. IGNORE — matches skip patterns or static file extensions
2. HOME — root URL or empty path
3. CATEGORY — path segment contains 'categor' OR equals 'products' (plural)
4. PRODUCT — path contains product URL signals
5. ABOUT — path contains about URL signals
6. CONTACT — path contains contact URL signals
7. PRODUCT — deep path heuristic (depth ≥ 2, nothing else matched, confidence 0.55)
8. IGNORE — fallback

Content Signal Thresholds

A page needs at least 2 content signals to be confirmed as that type. One word alone is not enough — 'packaging' could appear anywhere.

Product content signals include: add to cart, buy now, request a quote, price, per kg, MOQ, minimum order, HSN code, HS code, certifications, packaging, ingredients, composition, specification, technical data

About content signals include: founded, established, since, our company, manufacturer, exporter, supplier, ISO certified, GMP certified, years of experience, export to, clients worldwide

5. Content Cleaning

Three-Stage Cleaning Pipeline

The cleaner converts raw HTML into clean text the LLM can work with. It runs three strategies in priority order:

Stage	Method	When Used
0	inner_text (browser)	When browser already captured live DOM text (React/SPA sites). Bypasses trafilatura entirely. Most reliable for modern JS sites.
1	trafilatura	Primary extraction for traditional HTML sites (WordPress, static HTML). Best at identifying main content vs boilerplate.

2	BeautifulSoup fallback	When trafilatura returns nothing or fewer than 30 words. Strips noise tags manually then extracts remaining text.

Noise Removal

The following are stripped before extraction reaches the LLM:

HTML tags: script, style, noscript, nav, header, footer, aside, iframe, svg, canvas, form

CSS class patterns: cookie, banner, popup, modal, newsletter, subscribe, overlay, breadcrumb, pagination, sidebar, widget, social, share, related, ad-

Stop phrases (inner_text truncated at): Add To Your Strip Lineup, You may also like, Related products, Customers also bought, Recently viewed

Limits

Setting	Value	Purpose
min_word_count	30 words	Pages below this are considered empty — skip extraction
max_chars_for_extraction	10,000 chars	LLM token budget — product content rarely needs more
snippet_length	300 chars	Preview shown in Step 2 UI product list

6. LLM Extraction

Models Used

Purpose	Model	Settings
Product extraction	gpt-4o-mini	temperature=0.0 (deterministic), max_tokens=1000, JSON mode on
Company extraction	gpt-4o-mini	temperature=0.0, max_tokens=1000, JSON mode on
Semantic scoring	gpt-4o-mini	temperature=0.0, max_tokens=300

Product Fields Extracted

For each product page, the LLM extracts these fields:

Field	Type	Example
product_name	string	Organic Turmeric Powder
category	string	Nutraceutical
description	string	Full description as written on page — not paraphrased
price	string	$8.50/kg FOB Mumbai — raw text, not parsed
packaging	string	500g pouches, 25kg bulk bags

certifications	list	["GMP", "USDA Organic", "FSSAI"]
moq	string	500 kg
hs_code	string	0910.30 — numeric only, no labels
ingredients	string	Curcuma longa root, 100% pure
specifications	dict	{"curcumin_content": "5%", "mesh_size": "60"}
variants	list	["5% curcumin", "95% curcumin"]

Company Fields Extracted

From About Us and Contact pages (merged into one LLM call):

company_name, founded_year, tagline, country, city, address

business_type: Manufacturer / Trader / Exporter / Agent

industries_served, export_markets, export_experience

certifications, regulatory_approvals (FDA, FSSAI etc.)

manufacturing_capacity, annual_turnover, employee_count

contact_email, contact_phone, linkedin_url

Company extraction merges multiple pages (about + contact + home) into one LLM call so the model sees all available company information at once. Pages are sorted: ABOUT first, then CONTACT, then HOME.

7. Confidence Scoring

How Scoring Works

Every product gets a confidence score from 0 to 100. The score starts at 100 and points are deducted for each missing field. The score drives the traffic light display in the Step 2 UI.

Score Range	Tier	Meaning
60 – 100	🟢 HIGH	Ready for intelligence engine. All key fields (product name and description)present.
40-59	🟠 MEDIUM	Needs review. Some important fields missing. User prompted to fill gaps.
0 – 39	🔴 LOW	Too sparse. Enter manually or exclude from analysis.

Field Weights

Total of all weights = 100. Missing a field deducts that many points:

Field	Weight
product_name	30
description	20
category	15
price	8
packaging	5

certifications	5
moq	5
hs_code	4
ingredients	5
images	2
specifications	1

Quality Penalties

Additional penalties on top of field-presence deductions:

Extraction failed note in field: -40 points

JSON parse error: -30 points

Product name looks like a URL: -20 points

Description too short (< 30 chars): -10 points

Nutraceutical product has no certifications: -2 points

Semantic Scoring

For products scoring between 30 and 60, a second LLM call (gpt-4o-mini) evaluates quality — not just presence. It catches things rules cannot:

Category is present but is 'General' or 'N/A' — not useful

Description is present but is cookie banner text

Page extracted fine but is not actually a product page

Semantic scoring is skipped for products below 40 (already too low to rescue) and above 85 (already trusted). This saves tokens while adding value only where it matters.

8. FastAPI Endpoint

Running the Service

```
cd ~/Desktop/AI/website_dna
```

```
python -m uvicorn input_pipeline.api:app --reload --host <ip> --port 8000
```

Auto-generated interactive docs available at:

```
http://<host>:8000/docs
```

POST /pipeline/run

The main endpoint. Takes a website URL and returns structured product + company data ready for DB insertion.

Request Body

Field	Type	Required	Description
website_url	string	Yes	Website to crawl
company_id	string	Yes	Your DB company UUID — echoed back in response for linking
user_id	string	Yes	Who triggered this run
max_pages	int	No	Override default of 500
run_semantic	bool	No	Default true. Set false for faster

			dev runs

Response Structure

Field	Description
status	success | partial | failed
products[]	Array of extracted products — insert each as a row in product_master
company{}	Company data with preference_patch dict — use for UPSERT into company_preference
total_products	Total number of products found
high_confidence_count	Products scoring 80+ (green)
medium_confidence_count	Products scoring 50-79 (orange)
low_confidence_count	Products scoring below 50 (red)
crawl_stats{}	pages_fetched, pages_skipped, fetch_errors, product_pages, about_pages, contact_pages
errors[]	Non-fatal errors — pipeline always returns even on partial failure
elapsed_sec	Total pipeline duration in seconds

9. Running & Debugging

CLI Usage

```
cd ~/Desktop/AI/website_dna
```

```
# Basic run
```

```
python input_pipeline/run.py https://greenleafexports.in
```

```
# Fast test run (no semantic scoring, limit pages)
python input_pipeline/run.py https://example.com --no-semantic --max-pages 20

# Custom output file
python input_pipeline/run.py https://example.com --output my_results.json
```

Output files are saved to input_pipeline/output/ with timestamp prefix:

{domain}_{timestamp}_products.json — clean product list

{domain}_{timestamp}_company.json — company data

{domain}_{timestamp}_full.json — complete result with stats (for debugging)

Debug Scripts

Script	What It Does
debug_fetch.py <url>	Fetches one URL and prints HTML length, title, link count, and clean text — fastest way to diagnose why a page is not being extracted
debug_js.py <url>	Checks if JavaScript is executing properly — prints HTML length after 5s wait, body text, and whether React rendered content
debug_dump.py <url>	Saves full HTML to /tmp/debug_page.html and prints last 3000 chars — shows what was actually fetched

Common Issues and Fixes

Symptom	Cause	Fix
0 product pages found	SPA site — homepage returns no links	Orchestrator auto-tries /products, /product, /shop fallback paths
HTML length ~8,000 (too small)	React redirect splash screen — page redirects to itself after 4 seconds	wait_for_function waits until 'Redirecting' text disappears from DOM
All products score 0	clean_text is footer/cookie text not product content	inner_text fix: browser captures live DOM text directly
Category pages as products	URL classifier checked PRODUCT before CATEGORY	Fixed: CATEGORY checked first in priority order
Duplicate products	Same product extracted from listing page and detail page	Deduplication by product_name after extraction
ImportError on API start	AsyncOpenAI() called at module level without API key	Fixed: lazy _get_client() pattern in all extractor files
Service won't start (wrong Python)	System uvicorn used instead of conda env uvicorn	Always use: python -m uvicorn (not just uvicorn)

10. Environment Setup

Required Environment Variables

Create a .env file at ~/Desktop/AI/website_dna/.env:

```
OPENAI_API_KEY=sk-...       # GPT-4o-mini for extraction, GPT-4o for generation
PERPLEXITY_API_KEY=pplx-... # Perplexity Sonar for intelligence modules
```

Python Environment

```
conda activate product_insights   # Python 3.11
pip install fastapi uvicorn openai playwright trafilatura beautifulsoup4 python-dotenv pydantic
playwright install chromium        # Install browser for Playwright
```

Key Dependencies

Library	Purpose
playwright	Browser automation — fetches pages including React/SPA sites with full JS execution
trafilatura	Content extraction — strips HTML boilerplate, extracts main article/product content
beautifulsoup4	Fallback HTML parser — used when trafilatura returns insufficient content
openai	Client for both OpenAI (GPT-4o-mini/GPT-4o) and Perplexity Sonar (same SDK, different base_url)
pydantic	Data validation — all models (ProductData, CompanyData, RawPage etc.) are Pydantic BaseModel
fastapi + uvicorn	HTTP API server — exposes /pipeline/run endpoint for backend

	integration
python-dotenv	Loads API keys from .env file automatically